

1. Develop a C program that demonstrates how a Linux operating system executes a command entered by a user that 1. Accept a Linux command as input. 2. Create a child process using `fork()`. 3. Execute the command in the child process using an appropriate `exec()` system call. 4. Allow the parent process to wait for the child using `wait ()`. 5. Display the Process ID (PID) of both parent and child processes.

Using Linux terminal commands (`uname`, `lscpu`, `lsblk`, `ps`, `top`), investigate the relationship between hardware resources and operating system services. Prepare a report explaining how the OS abstracts CPU, memory, storage, and I/O devices.

exp1.c

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()
{
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }
    else if (pid == 0)
```

```
{  
    printf("Child Process\n");  
    printf("Child PID: %d\n", getpid());  
    printf("Parent PID: %d\n", getppid());  
  
    execlp(command, command, NULL);  
  
    perror("exec failed");  
    exit(1);  
}  
else  
{  
    printf("Parent Process\n");  
    printf("Parent PID: %d\n", getpid());  
    printf("Child PID: %d\n", pid);  
  
    wait(NULL);  
  
    printf("Child process completed.\n");  
}  
  
return 0;  
}
```

Output :

```

charanteja_a@charan: $ nano pro1.c
charanteja_a@charan: $ gcc pro1.c -o pro1
charanteja_a@charan: $ ./pro1
Enter a Linux command: ls
Parent Process
Parent PID: 2813
Child PID: 2816
Child Process
Child PID: 2816
Parent PID: 2813
OSSP      exp1.c  expe2   expe3.c  fork     loop    main.c  p1.c    p2      p3.c    p5      p6.c    p9      pc.c.save  pro_con  read.c
example.c expe   expe2.c filecall fork.c   loop.c  myshell p10     p2.c    p4      p5.c    p7      p9.c    pro1      pro_con.c week2s.c
exp1      expe.2 expe3   filecall input.txt main     p1      p10.c   p3      p4.c    p6      p7.c    pc.c      pro1.c    read
Child process completed.

```

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

fork_process.c

```

#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>

#include <stdlib.h>

int main()

{

    pid_t pid;

    printf("Before fork():\n");

    printf("Process PID : %d\n", getpid());

    printf("Parent PPID : %d\n", getppid());


    pid = fork();


    if (pid < 0)

    {

        perror("fork failed");

        return 1;

    }

```

```
else if (pid == 0)
{
    // Child process

    printf("\n--- CHILD PROCESS ---\n");
    printf("Child PID   : %d\n", getpid());
    printf("Child PPID  : %d\n", getppid());
    printf("Child State : Running\n");

    sleep(5);

    printf("Child State : Terminating\n");
    exit(0);
}

else
{
    // Parent process

    printf("\n--- PARENT PROCESS ---\n");
    printf("Parent PID   : %d\n", getpid());
    printf("Parent PPID  : %d\n", getppid());
    printf("Child PID    : %d\n", pid);
    printf("Parent State : Running\n");

    printf("\nParent waiting for child...\n");
    wait(NULL);

    printf("Child terminated.\n");
    printf("Parent State : Running\n");
    printf("Parent State : Terminated after completion\n");
}
```

```
return 0;
}
```

Output :

```
child process completed.
charanteja_a@charan:~$ nano pro2.c
charanteja_a@charan:~$ gcc pro2.c -o pro2
charanteja_a@charan:~$ ./pro2
Before fork():
Process PID   : 2865
Parent PPID   : 2738

--- PARENT PROCESS ---
Parent PID    : 2865
Parent PPID   : 2738
Child PID     : 2866
Parent State  : Running

Parent waiting for child...
--- CHILD PROCESS ---
Child PID     : 2866
Child PPID    : 2865
Child State   : Running
Child State   : Terminating
Child terminated.
Parent State  : Running
Parent State  : Terminated after completion
```

Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

exp3.c

```
#include <stdio.h>

#include <unistd.h>

#include <sys/types.h>

int main()
{
    pid_t pid;

    printf("Before fork()\n");
    printf("Process PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());
    }
}
```

```

        sleep(10);

        printf("Child process terminating...\n");
    }
    else
    {
        printf("\nParent Process\n");
        printf("PID : %d\n", getpid());
        printf("PPID : %d\n", getppid());

        sleep(5);

        printf("Parent process terminating...\n");
    }

    return 0;
}

```

Output:

```

charanteja_a@charan:~$ nano pro3.c
charanteja_a@charan:~$ gcc pro3.c -o pro3
charanteja_a@charan:~$ ./pro3
Before fork()
Process PID: 2906

Parent Process
PID : 2906
PPID : 2738

Child Process
PID : 2907
PPID : 2906
Parent process terminating...

```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

exp4.c

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
    int i;
```

```
    pid_t pid;
```

```
    for (i = 0; i < 3; i++)
```

```
    {
```

```
        pid = fork();
```

```
        if (pid < 0)
```

```
        {
```

```
            perror("fork failed");
```

```
            exit(1);
```

```
        }
```

```
        if (pid == 0)
```



```

{
    printf("Child %d: PID = %d, PPID = %d\n",
        i + 1, getpid(), getppid());

    sleep(i + 1);

    printf("Child %d completed.\n", i + 1);
    exit(0);
}
}

for (i = 0; i < 3; i++)
{
    wait(NULL);
    printf("Parent collected a child.\n");
}

printf("All children completed.\n");

return 0;
}

```

OUTPUT:

```
charanteja_a@charan:~$ nano pro4.c
charanteja_a@charan:~$ gcc pro4.c -o pro4
charanteja_a@charan:~$ ./pro4
Child 1: PID = 2956, PPID = 2955
Child 2: PID = 2957, PPID = 2955
Child 3: PID = 2958, PPID = 2955
Child 1 completed.
Parent collected a child.
Child 2 completed.
Parent collected a child.
Child 3 completed.
Parent collected a child.
All children completed.
```